Clearway design

Scoring methodology, where AI is used, key tradeoffs

Clearway screens US airports for terminal investment. A Next.js chat on Vercel calls a Hono API on AWS Lambda, where Claude runs seven typed tools over Postgres on Neon. A daily job loads the BTS traffic API; FAA and NOAA are read live per question. Full detail: engineering notes.

Scoring methodology

Measures use the official definitions and are computed in tested TypeScript over the same 12 months for every airport (May 2025 to April 2026): growth = last 12 months ÷ the 12 before − 1; load factor = passengers ÷ seats (summed, not averaged); supply gap = passenger growth − seat growth; delay rate = arrivals 15+ minutes late ÷ arrivals; long-haul share = departures of 4,000 km or more ÷ all departures.

Expansion screening score (0 to 100). Each component earns weight × (value − low) ÷ (high − low), clamped to its range:

Component	Range	Weight
Demand growth (passengers)	−5% to +15%	35
Seat occupancy (load factor)	70% to 95%	30
Delay pressure (delay rate)	0% to 35%	20
Supply gap	0 to +5 pts	15

Fixed ranges keep a score independent of which airports are compared. Airports under 100,000 passengers or missing a component are listed as not scored, never zero-filled. A high score means "worth a terminal-capacity study", not a return: public data has no costs or revenues. Weights are reasoned, not calibrated; partial weight changes are re-run as labeled scenarios.

Unmet demand cannot be counted, so four signals are tested: planes fuller than the national load factor for the same months (a measured benchmark rather than a chosen threshold); supply gap above zero; delay rate 25%+ or system share above 40%; a documented capacity limit. Met signals give the level: 0 to 1 low, 2 moderate, 3 to 4 high.

Where and how AI is used

Claude (Sonnet, via the Vercel AI SDK) does three things: understands the question, chooses and parameterizes tools, and explains the results. It never computes or recalls a number. Tools validate their inputs (Zod, checked airport codes, no free SQL) and return each figure pre-rounded inside an evidence envelope with period, sources, assumptions and confidence. Follow-ups re-run the tool with the changed argument. Guardrails: at most 6 steps per answer; tool output is data, never instructions; out-of-scope questions (ROI, costs) are declined. An evaluation of the exam questions and follow-ups fails any answer that states a number no tool returned (8 of 8 pass). No RAG or agent framework: answers are calculations over tables, and one tool loop is simpler to test and bound.

Key tradeoffs

Decision	Why	Cost
Screen demand, not profit	Public data shows demand pressure but no costs or revenues	No ROI figure; stated in every relevant answer
BTS API stored daily, not fetched per question	Data is monthly; one ranking needs many paged calls; survives outages	Up to a day behind BTS (BTS itself is monthly)
Two official files besides the API	No free API has route distances or delay causes	Refreshed by re-import, not automatically
Fixed score ranges	A score does not move when other airports are added	Ranges are judgment, documented and adjustable
Serverless: Lambda and Neon	Pay per request, scales to zero, no servers to run	Cold starts; the rate limit is per instance